

DOKUMENTACJA TECHNICZNA

WATIR IoT

Watering Automation Technology for Indoor Plants

Zespół Projektowy:

Wojciech Stochmiałek - Hardware
Bartłomiej Raj - Sieci / Lider Zespołu
Marcel Podlecki - Aplikacja Mobilna (UI/UX)
Bartłomiej Jedyk - Backend / Bazy Danych
Dominik Pokorski - Aplikacja Mobilna (Logika)

Opiekun projektu: dr inż. Jacek Nazdrowicz K22

Rok akademicki: 2025/2026

Repozytorium kodu źródłowego: https://github.com/BaDaQu/WATIR_IoT

Spis treści

Spis treści	2
1. Wstęp i cel projektu	4
1.1. Tło problemowe	4
1.2. Analiza rynku i rozwiązań alternatywnych	4
1.3. Cel i proponowane rozwiązanie	4
2. Geneza i ewolucja architektury projektu	5
2.1. Wersja Proof of Concept (PoC v0)	5
2.2. Iteracje pośrednie (v1.0–v1.3)	6
2.3. Architektura docelowa (v2.0)	7
3. Użyte technologie (Tech Stack)	7
3.1. Frontend - Aplikacja mobilna	7
3.2. Backend - Warstwa chmurowa	7
3.3. Baza danych	8
3.4. Infrastruktura	8
3.5. Hardware - Edge Control	8
4. Architektura systemu i struktura repozytorium	8
4.1. Przegląd architektury trójwarstwowej	8
4.2. Warstwa sprzętowa (Edge Control)	9
4.3. Warstwa chmurowa	10
4.4. Warstwa aplikacyjna (mobile)	10
4.5. Struktura repozytorium kodu	10
5. Protokół komunikacji (REST API)	11
5.1. Telemetria (Urządzenie → Backend)	11
5.2. Sterowanie zdalne (Backend/Aplikacja → Urządzenie)	12
5.3. Zarządzanie profilami roślin (CRUD)	12
6. Hybrydowy routing (Dual-Path) i odporność na awarie	13
6.1. Ścieżka chmurowa (telemetria i konfiguracja)	13
6.2. Ścieżka lokalna (sterowanie bezpośrednie)	13
6.3. Odporność na błędy i praca w trybie offline (Edge Autonomy)	13
7. Główne funkcjonalności i logika sterująca	14
7.1. Tryb automatyczny (AUTO)	14
7.2. Sterowanie zdalne (aplikacja mobilna / joystick)	15
7.3. Zapis telemetrii i analityka historyczna	15
8. Baza danych	15
8.1. Tabela telemetry_logs	15
8.2. Tabela plant_profiles	16
9. Środowisko lokalne - instrukcja uruchomienia	16
9.1. Wymagania wstępne	16

9.2. Uruchomienie infrastruktury backendowej	16
9.3. Instalacja aplikacji mobilnej.....	16
9.4. Wgrywanie firmware na Arduino UNO R4 WiFi.....	16
10. Testowanie i walidacja systemu	17
10.1. Kalibracja czujników.....	17
10.2. Testy integracyjne i end-to-end	17
10.3. Walidacja stabilności długoterminowej	17
11. Napotkane problemy i rozwiązania inżynierskie	18
12. Kluczowe innowacje, osiągnięcia i podział prac zespołu	19
12.1. Innowacje wykraczające poza minimum projektu.....	19
12.2. Podział prac w zespole	19
13. Harmonogram projektu i zarządzanie zadaniami	19
13.1. Kamienie milowe.....	20
14. Znane ograniczenia i możliwości dalszego rozwoju	20
14.1. Znane ograniczenia	20
14.2. Możliwości dalszego rozwoju	20
15. Podsumowanie i wnioski końcowe	21
16. Literatura i źródła.....	21

1. Wstęp i cel projektu

1.1. Tło problemowe

Utrzymanie systematycznej pielęgnacji roślin domowych stanowi istotne wyzwanie dla osób aktywnych zawodowo oraz często podróżujących. Nieregularność podlewania - wynikająca z wyjazdów służbowych, urlopów lub braku czasu na co dzień - prowadzi do dwóch skrajnych, negatywnych scenariuszy: przesuszenia podłoża lub przelania skutkującego gniciem systemu korzeniowego. Problem ten dotyczy zarówno hobbystów posiadających pojedyncze okazy, jak i zaawansowanych kolekcjonerów prowadzących rozbudowane uprawy typu „urban jungle”, gdzie liczba doniczek wymagających indywidualnego nadzoru znacząco utrudnia manualną kontrolę. Nieprawidłowa pielęgnacja prowadzi nie tylko do strat finansowych przy rzadkich okazach, lecz również zaburza estetykę i mikroklimat pomieszczenia, a doraźne prośby o pomoc kierowane do sąsiadów czy współlokatorów w praktyce bywają zawodne.

1.2. Analiza rynku i rozwiązań alternatywnych

Grupą docelową systemu jest osoba aktywna zawodowo, często podróżująca, o typowo konsumenckim poziomie kompetencji technicznych - system nie może wymagać od użytkownika wiedzy inżynierskiej, programistycznej ani umiejętności konfiguracji sieciowej. Dostępne obecnie na rynku rozwiązania alternatywne cechują się istotnymi ograniczeniami, zestawionymi w poniższej analizie:

- **Kule nawadniające (Aqua Globes)** - rozwiązanie tanie i powszechnie dostępne, jednak całkowicie pozbawione precyzji dozowania. Stwarzają wysokie ryzyko przelania lub przesuszenia rośliny; brak jakiegokolwiek monitoringu, kontroli zdalnej czy powiadomień o stanie systemu.
- **Dedykowane „Smart Doniczki”** - rozwiązania oferujące monitoring i automatyzację, ale w modelu 1:1, w którym każda pojedyncza, kosztowna doniczka jest przypisana na stałe do obsługi tylko jednej rośliny. Przy próbie skalowania na całą domową kolekcję koszt wdrożenia rośnie liniowo i szybko staje się ekonomicznie nieuzasadniony.

Przewagą konkurencyjną systemu WATIR IoT względem powyższych rozwiązań jest architektura 1 stacja bazowa : wiele doniczek (1:Wielu), oparta na ruchomym ramieniu sterowanym w płaszczyźnie dwuwymiarowej (mechanizm Pan-Tilt). Pojedyncza stacja bazowa umożliwia precyzyjne podlewanie wielu, indywidualnie rozmieszczonych donic znajdujących się w zasięgu urządzenia, co stanowi ekonomiczną optymalizację kosztu wdrożenia systemu smart home w skali całej domowej kolekcji roślin - bez konieczności instalowania odrębnego zestawu czujników i elementów wykonawczych przy każdej roślinie z osobna.

1.3. Cel i proponowane rozwiązanie

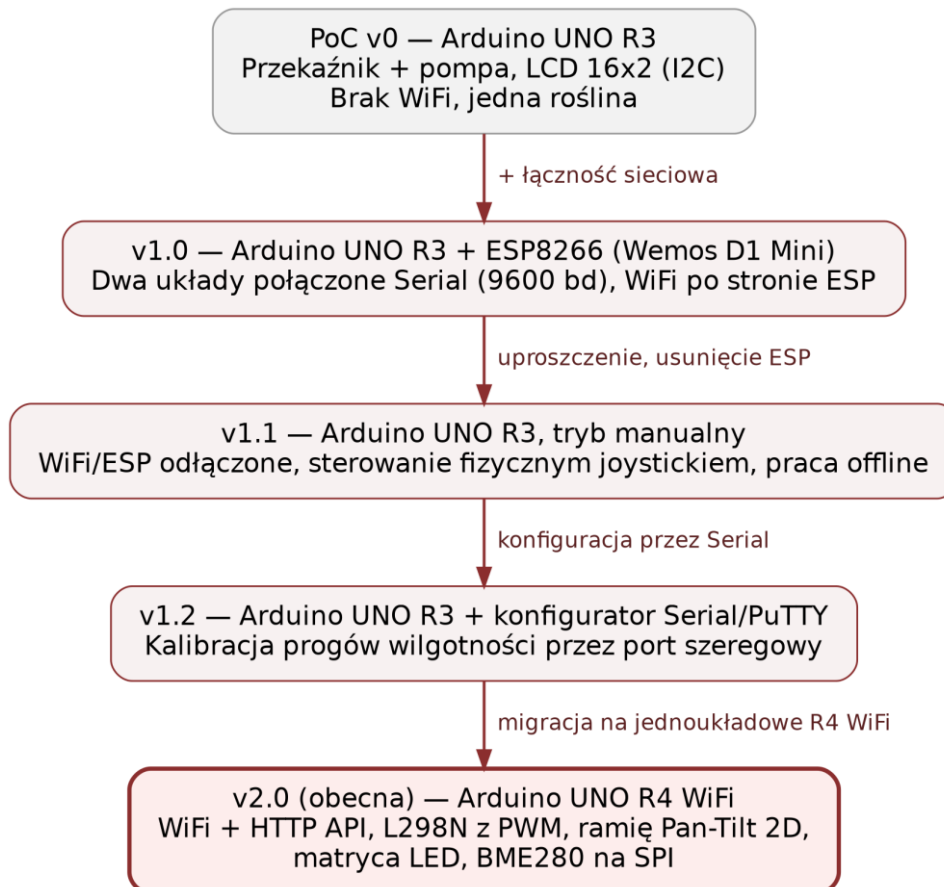
Głównym celem projektu WATIR IoT jest zaprojektowanie, wykonanie oraz oprogramowanie zintegrowanego systemu wbudowanego, który bez udziału człowieka monitoruje wilgotność gleby oraz parametry mikroklimatu, a następnie precyzyjnie podlewa wybrane rośliny domowe, raportując jednocześnie zebrane dane do chmury obliczeniowej. System realizuje pełen cykl przetwarzania:

- akwizycja danych środowiskowych i glebowych z czujników sprzętowych,
- podjęcie decyzji sterującej na podstawie zdefiniowanego dla danej rośliny profilu (progu wilgotności),
- działanie wykonawcze - pozycjonowanie ramienia Pan-Tilt nad wybraną doniczką i aktywacja pompy wodnej,
- raportowanie stanu systemu do chmury obliczeniowej (PostgreSQL za pośrednictwem API Express.js),
- zdalny wgląd i kontrola użytkownika za pośrednictwem natywnej aplikacji mobilnej na system Android, niezależnie od lokalizacji użytkownika.

Projekt zaprojektowano zgodnie z metodologią pracy zwinnej (Agile) z wykorzystaniem GitHub Projects jako centralnego narzędzia koordynacji zadań zespołu pięcioosobowego, co opisano szerzej w rozdziale 13.

2. Geneza i ewolucja architektury projektu

System WATIR IoT w obecnym kształcie jest wynikiem kilkietapowej ewolucji architektonicznej - od prostego, jednoukładowego prototypu laboratoryjnego (Proof of Concept), poprzez pośrednie iteracje sprzętowe, aż po docelową, jednoukładową platformę IoT opartą na Arduino UNO R4 WiFi. Prześledzenie tej ścieżki rozwoju ma istotną wartość inżynierską - dokumentuje decyzje projektowe podjęte w odpowiedzi na napotkane ograniczenia sprzętowe i programowe.

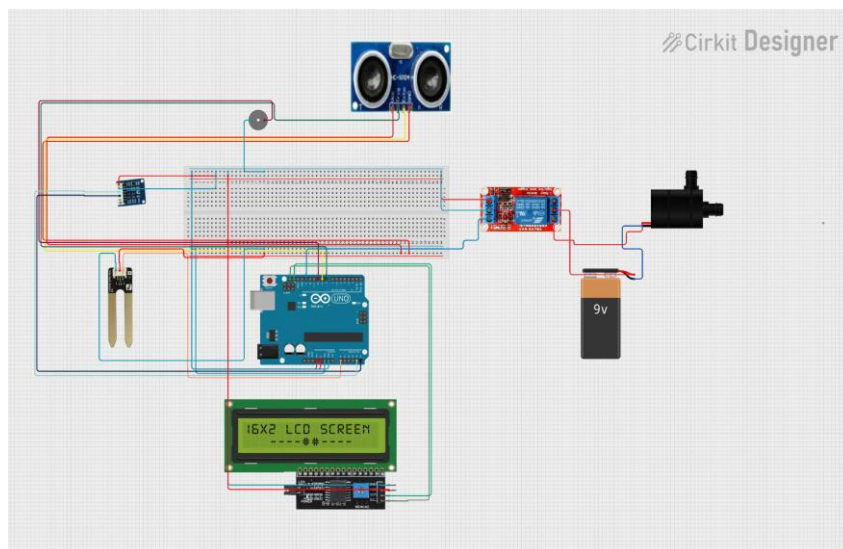


Rysunek 1 Ewolucja architektury sprzętowej projektu WATIR / WATIR IoT.

2.1. Wersja Proof of Concept (PoC v0)

Punktem wyjścia był laboratoryjny prototyp zrealizowany na platformie Arduino UNO R3, pozbawiony łączności bezprzewodowej. Architektura sprzętowa opierała się na pojedynczym mikrokontrolerze ATmega328P, module przełącznikowym 1-kanalowym 5V sterującym miniaturową pompą zanurzeniową, czujniku ultradźwiękowym HC-SR04 zabezpieczającym pompę przed pracą „na sucho”, rezystancyjnym czujniku wilgotności gleby oraz module BME280 podłączonym magistralą I2C do monitoringu mikroklimatu. Interfejs użytkownika stanowił wyświetlacz LCD 16x2 z konwerterem I2C. System obsługiwał wyłącznie jedną roślinę.

Wartość progowa `moistureThreshold` została w tej wersji ustalona eksperymentalnie na poziomie surowego odczytu ADC równym 200 (w zakresie 0–1023), a algorytm sterujący, oparty na modelu Single Loop Control z priorytetyzacją bezpieczeństwa, blokował wykonanie dalszej części kodu pętlą `while (distance >= 12)` do czasu uzupełnienia zbiornika.



Rysunek 2 Schemat połączeń prototypu Proof of Concept (Cirkuit Designer) - Arduino UNO R3, przekaźnik, pompa zanurzeniowa, HC-SR04, LCD I2C.



Rysunek 3 Fizyczna realizacja prototypu PoC - pojedyncza roślina, stacjonarny układ elektroniki w obudowie ochronnej oraz zewnętrzny zbiornik wody.

Wersja ta nie posiadała mechanizmu pozycjonowania - wylot wody realizowany był statycznym wężykiem silikonowym skierowanym bezpośrednio nad pojedynczą doniczkę, a stan systemu odczytywano lokalnie z ekranu LCD (Rys. poniżej), bez możliwości zdalnego podglądu.



Rysunek 4 Interfejs LCD prototypu PoC - odczyty temperatury (T), wilgotności powietrza (H), wilgotności gleby (G) oraz poziomu wody (W).

2.2. Iteracje pośrednie (v1.0–v1.3)

W kolejnym etapie rozwoju architekturę rozszerzono o łączność sieciową oraz mechanizm pozycjonowania przestrzennego, wprowadzając istotne modyfikacje względem pierwotnych, laboratoryjnych założeń:

- **v1.0 - architektura dwuukładowa:** Arduino UNO R3 połączone portem szeregowym (Serial RX/TX, 9600 bd) z modułem ESP8266 (Wemos D1 Mini). Arduino odpowiadało za warstwę wykonawczą (serwomechanizmy Pan-Tilt, odczyt czujników, przekaźnik pompy), natomiast ESP8266 pełniło rolę asynchronicznego serwera HTTP w sieci lokalnej oraz klienta raportującego telemetrię do chmury.
- **v1.1 - tryb manualny:** w celu wyeliminowania niestabilności komunikacji WiFi/ESP i uzyskania w pełni działającego systemu offline, moduł ESP8266 odłączono, a sterowanie ramieniem oparto wyłącznie na fizycznym joysticku. Główna pętla programu została uproszczona do trzech zadań: aktualizacji pozycji serw, cyklicznego odczytu czujników oraz odświeżania wyświetlacza LCD.

- **v1.2 - konfigurator Serial/PuTTY:** dodano interaktywny konfigurator uruchamiany w fazie setup(), umożliwiający zdalne (poprzez port szeregowy) ustawienie progów wilgotności gleby, temperatury i wilgotności powietrza oraz kalibrację pozycji ramienia „na żywo” - bez konieczności ponownego wgrywania firmware’u.
- **v1.3 - tryb offline:** zamieniono konfigurator PuTTY na wybór opcji po nieudanej autoryzacji sieci Wi-Fi. Użytkownik za pomocą ekranu LCD i joysticka ustawia progi wilgotności gleby, temperatury, wilgotności powietrza oraz pozycje roślin. Takie profile zapisywane są jako Roślina [numer].

Na tym etapie zaobserwowano i rozwiązano również pierwsze poważne problemy sprzętowe - telepanie się serwomechanizmów spowodowane zakłóceniami elektromagnetycznymi anteny WiFi oraz konfliktem przerwań między bibliotekami obsługującymi port szeregowy i bibliotekę Servo, a także spadki napięcia przy jednoczesnym starcie pompy i serw, skutkujące niekontrolowanymi resetami mikrokontrolera. Rozwiązania tych problemów opisano szczegółowo w rozdziale 11.

2.3. Architektura docelowa (v2.0)

Ostatecznym etapem ewolucji była migracja na jednoukładową platformę Arduino UNO R4 WiFi (mikrokontroler Renesas RA4M1 ze zintegrowanym koprocesorem sieciowym ESP32-S3), która przejęła funkcje obu wcześniej rozdzielonych układów. Komunikacja Serial między płytkami została całkowicie wyeliminowana, a WiFi, HTTP API, sterowanie serwami, pompą, wyświetlaczem oraz matrycą LED zostały zintegrowane w jednym, spójnym firmware (WATIR-ARDUINO.ino), który rozrósł się z pierwotnych 131 do ponad 860 linii kodu. Decyzja o migracji została podjęta dynamicznie w trakcie realizacji Milestone 3 harmonogramu projektu, co GitHub Projects - jako centralne narzędzie zarządzania zadaniami - pozwoliło sprawnie odzwierciedlić w strukturze zadań zespołu, bez zakłócenia ciągłości prac pozostałych podzespołów (aplikacji mobilnej i backendu).

W przypadku rozbieżności pomiędzy niniejszą dokumentacją a stanem faktycznym kodu źródłowego, za nadrzędny uznaje się kod repozytorium - dziennik zmian firmware (branch feature/firmware-r4-wifi-migration, wersja 2.0) stanowi jedynie archiwum procesu decyzyjnego.

3. Użyte technologie (Tech Stack)

3.1. Frontend - Aplikacja mobilna

Technologia	Zastosowanie i uzasadnienie techniczne
Kotlin	Natywny, w pełni wspierany przez Google język programowania dla platformy Android, zapewniający bezpieczeństwo typów oraz zwięzłą składnię ograniczającą liczbę błędów runtime.
Jetpack Compose	Deklaratywny framework UI umożliwiający szybkie i wydajne renderowanie nowoczesnego interfejsu użytkownika bez konieczności zarządzania cyklem życia widoków w sposób imperatywny.
Retrofit	Biblioteka HTTP client mapująca zapytania REST na interfejsy Kotlin (WatirAPI), wykorzystywana zarówno do komunikacji z chmurowym API Express.js, jak i bezpośrednio z lokalnym adresem IP Arduino R4 WiFi.

3.2. Backend - Warstwa chmurowa

Technologia	Zastosowanie i uzasadnienie techniczne
Node.js	Asynchroniczne, jednowątkowe środowisko uruchomieniowe JavaScript oparte na pętli zdarzeń (event loop), predestynowane do obsługi wielu równoległych, krótkotrwałych zapytań telemetrycznych charakterystycznych dla systemów IoT.
Express.js	Minimalistyczny framework webowy odpowiedzialny za routing zapytań API REST (m.in. /api/telemetry, /api/plants), walidację ładunków JSON oraz proxy komend sterujących do lokalnego IP stacji bazowej.

3.3. Baza danych

Technologia	Zastosowanie i uzasadnienie techniczne
PostgreSQL	Relacyjny system zarządzania bazą danych (RDBMS) zapewniający spójność transakcyjną (ACID), wykorzystany do przechowywania logów telemetrycznych (telemetry_logs) oraz profili roślin (plant_profiles).

3.4. Infrastruktura

Technologia	Zastosowanie i uzasadnienie techniczne
Docker / Docker Compose	Konteneryzacja serwera Express.js oraz bazy PostgreSQL w jednej, deklaratywnej, łatwo uruchamialnej sieci kontenerów; polityka restart:always zapewnia automatyczne podniesienie serwera po błędzie krytycznym.

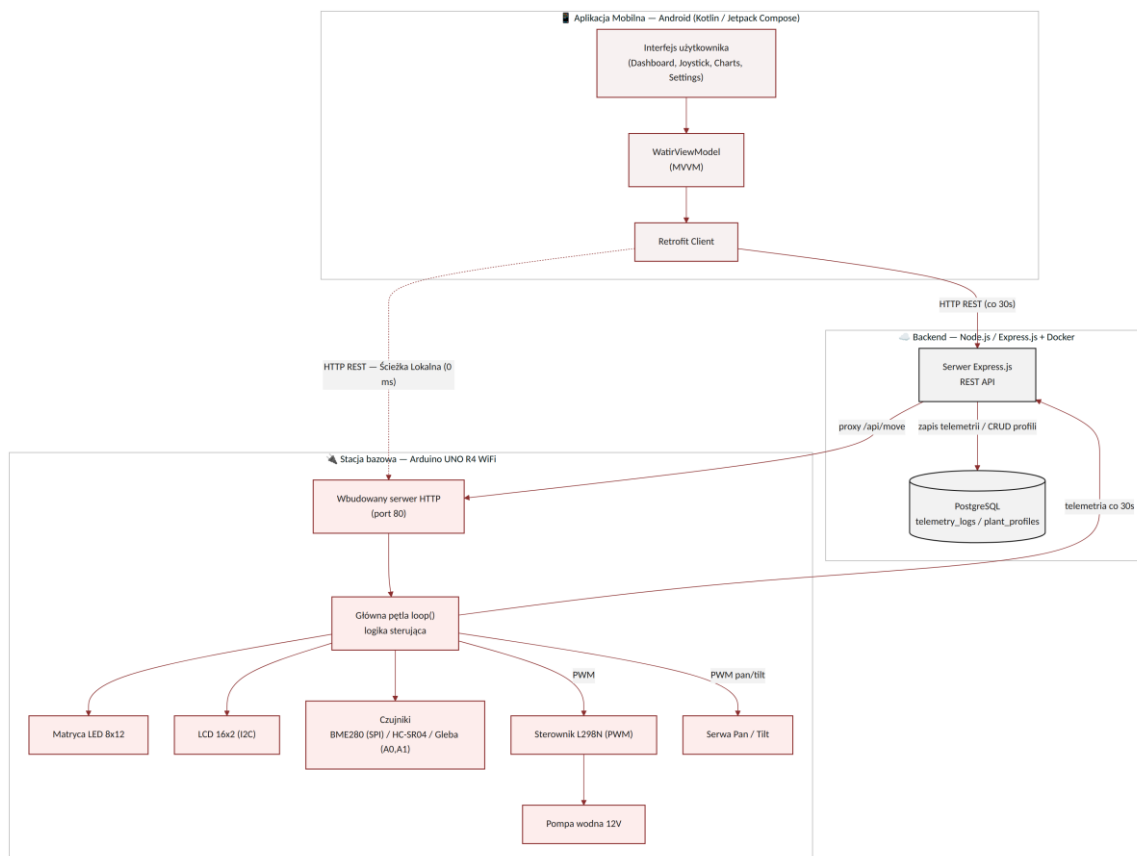
3.5. Hardware - Edge Control

Technologia	Zastosowanie i uzasadnienie techniczne
Arduino UNO R4 WiFi	Centralna jednostka decyzyjna systemu (Renesas RA4M1 + koprocessor ESP32-S3), integrująca WiFi, logikę sterującą czasu rzeczywistego oraz wbudowany serwer HTTP w jednym układzie.
C++ / Arduino IDE	Natywne programowanie mikrokontrolera zapewniające pełną kontrolę nad zasobami sprzętowymi oraz determinizm czasowy operacji sterujących serwomechanizmami i pompą.
WiFiS3.h	Dedykowana biblioteka komunikacji sieciowej dla Arduino UNO R4 WiFi, obsługująca zarówno wysyłkę telemetry, jak i wbudowany serwer HTTP nasłuchujący na porcie 80.
ArduinoJson	Biblioteka do parsowania i budowania ładunków JSON wymienianych z backendem oraz aplikacją mobilną (kontrakt danych ustalony w Issue #9).
Adafruit BME280 / Unified Sensor	Obsługa cyfrowego czujnika temperatury, wilgotności powietrza i ciśnienia, komunikującego się z mikrokontrolerem magistralą SPI.
LiquidCrystal I2C	Sterowanie lokalnym wyświetlaczem LCD 16x2 podłączonym przez konwerter I2C.
Servo / EEPROM / SPI / Wire	Biblioteki wbudowane w Arduino IDE obsługujące odpowiednio: sterowanie serwomechanizmami PWM, nieulotny zapis pozycji ramienia i konfiguracji, komunikację SPI (BME280) oraz I2C (LCD).
Arduino_LED_Matrix	Obsługa wbudowanej w płytce R4 WiFi matrycy diagnostycznej LED 8x12, wykorzystywanej jako panel statusu systemu.

4. Architektura systemu i struktura repozytorium

4.1. Przegląd architektury trójwarstwowej

System WATIR IoT zaprojektowano w trójwarstwowej architekturze hybrydowej, łączącej elastyczność chmury obliczeniowej z zerowymi opóźnieniami lokalnego sterowania mechatronicznego: warstwę sprzętową (Edge Control) opartą na Arduino UNO R4 WiFi, warstwę chmurową (Node.js/Express.js + PostgreSQL, skonteneryzowaną w Dockerze) oraz warstwę użytkową (natywna aplikacja Android). Pełny przepływ danych pomiędzy warstwami przedstawiono na Rys. poniżej.



Rysunek 5 Architektura systemu WATIR IoT - warstwa mobilna, chmurowa oraz sprzętowa (Edge) wraz z kierunkami przepływu danych.

4.2. Warstwa sprzętowa (Edge Control)

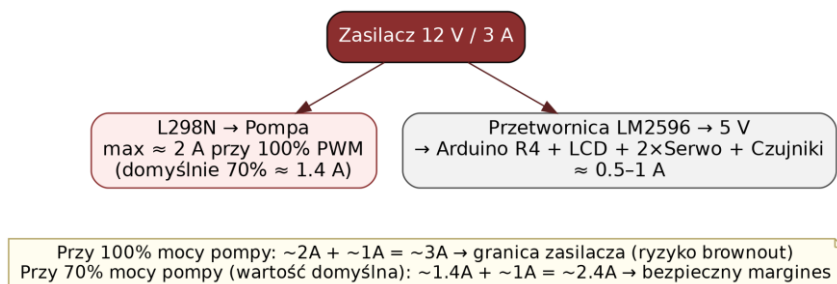
Centralną jednostką sterującą jest Arduino UNO R4 WiFi, do którego podłączono osiem głównych podzespołów funkcjonalnych: czujnik środowiskowy BME280 (magistrala SPI), czujnik ultradźwiękowy poziomu wody HC-SR04, dwa analogowe czujniki wilgotności gleby (obsługujące dwie niezależne rośliny), sterownik silnikowy L298N regulujący pracę pompy wodnej 12V sygnałem PWM, dwa serwomechanizmy tworzące ramię Pan-Tilt, wyświetlacz LCD 16×2 (I2C) oraz wbudowaną w płytkę R4 WiFi matrycę diagnostyczną LED 8×12. Szczegółową mapę wyprowadzeń przedstawia Tabela poniżej.

Pin	Funkcja	Moduł / uwagi
2	Przycisk joysticka	INPUT_PULLUP
3	L298N ENA (PWM)	Regulacja mocy pompy 0–100%
4	L298N IN2	Kierunek pompy (–)
5	L298N IN1	Kierunek pompy (+)
6	Serwo X (pan)	Ramię robota
7	HC-SR04 Echo	Czujnik poziomu wody
8	HC-SR04 Trig	Czujnik poziomu wody
9	Serwo Y (tilt)	Ramię robota
10	BME280 CS	SPI Chip Select
11 / 12 / 13	SPI MOSI / MISO / SCK	Magistrala BME280
A0 / A1	Czujnik gleby 2 / 1	Wejście analogowe
A2 / A3	Joystick X / Y	Wejście analogowe
SDA / SCL	LCD 16×2	Magistrala I2C

Tabela 1 Mapa pinów Arduino UNO R4 WiFi

Zasilanie warstwy wykonawczej rozdzielono elektrycznie od zasilania logiki mikrokontrolera w celu ochrony przed spadkami napięcia (brownout) przy jednoczesnym starcie pompy i serwomechanizmów - zasilacz 12V/3A zasila

bezpośrednio sterownik L298N, natomiast przetwornica step-down LM2596 dostarcza stabilne 5V dla Arduino, LCD, serw i czujników, przy zachowaniu wspólnej masy (Common Ground) całego układu.



Rysunek 6 Budżet mocy zasilania - rozdział obciążenia między sterownik pompy L298N a logikę sterującą

⚠ UWAGA: Domyślna moc pompy ustalono na 70% (a nie 100%) sygnału PWM. Przy pełnej mocy sumaryczny pobór prądu (pompa + logika) zbliża się do granicy wydajności zasilacza 3A, co grozi brownoutem; przy 70% zapewniony jest bezpieczny margines ($\sim 2,4 A$), a minimalna praktyczna wartość mocy to ok. 40–50% - poniżej tego progu silnik pompy nie ma momentu potrzebnego do rozruchu i jedynie emituje słyszalne buczenie (PWM ≈ 490 Hz).

4.3. Warstwa chmurowa

Serwer Express.js działający w kontenerze Docker pełni rolę pośrednika między stacją bazową a aplikacją mobilną. Kluczowym mechanizmem warstwy chmurowej jest automatyczne mapowanie adresów IP urządzeń (auto-discovery) - backend zapamiętuje w pamięci procesu rzeczywisty, lokalny adres IP nadawcy telemetry (przesyłany przez Arduino w polu ładunku JSON), dzięki czemu aplikacja mobilna nie musi znać adresu stacji bazowej z góry: komendy trafiające do chmury są transparentnie przekierowywane na właściwe urządzenie.

Dodatkowo backend zaimplementowano z uwzględnieniem odporności na awarie - w przypadku krytycznego błędu połączenia z PostgreSQL proces kończy działanie (process.exit(1)), co w połączeniu z polityką restart: always zdefiniowaną w Docker Compose skutkuje automatycznym, samoczynnym podniesieniem kontenera.

4.4. Warstwa aplikacyjna (mobile)

Aplikacja mobilna została zaprojektowana zgodnie ze wzorcem architektonicznym MVVM (Model-View-ViewModel) i podzielona na epiki funkcjonalne odpowiadające ekranom: dashboard, joystick, charts oraz settings, z centralnym komponentem WatirViewModel zarządzającym stanem UI oraz asynchronicznym pobieraniem danych. Warstwa sieciowa (network) zawiera fabrykę klienta Retrofit (APIClients) oraz interfejs WatirAPI definiujący dostępne endpointy zarówno chmurowego API, jak i lokalnego API stacji bazowej. Szczegółowy opis funkcjonalny warstwy mobilnej przedstawiono w rozdziale 7.

4.5. Struktura repozytorium kodu

Repozytorium projektu podzielono na cztery główne moduły funkcjonalne (aplikacja mobilna, backend, firmware, dokumentacja) oraz katalog narzędzi pomocniczych, zgodnie z poniższym drzewem katalogów:

```

WATIR_IoT/
├── app/                # Aplikacja mobilna (Android Studio / Kotlin)
│   ├── src/main/java/.../watir_iot_app/
│   │   ├── MainActivity.kt    # Punkt wejścia, Scaffold, Navigation
│   │   ├── core/navigation/Screen.kt # Definicje ekranów
│   │   ├── data/model/Models.kt  # Modele danych (Telemetry, Profiles)
│   │   ├── feature/joystick/JoystickScreen.kt
│   │   ├── feature/settings/SettingsScreen.kt
│   │   ├── network/{APIClients,WatirAPI}.kt
│   │   └── viewmodel/WatirViewModel.kt
└──
    
```

```

├── backend/          # Serwer Express.js + baza danych
│   ├── docker-compose.yml # Definicja kontenera PostgreSQL
│   ├── Dockerfile      # Budowa kontenera serwera Express
│   ├── index.js         # API endpoints, retencja, routing
│   ├── init.sql         # Inicjalizacja schematu (tabele, indeksy)
│   └── package.json      # express, cors, pg, dotenv
│
├── firmware/
│   ├── WATIR-ARDUINO/    # Firmware Arduino UNO R4 WiFi (aktualny)
│   │   ├── WATIR-ARDUINO.ino
│   │   ├── Sensors.cpp/.h # BME280 (SPI), HC-SR04, gleba
│   │   ├── ServoControl.cpp/.h # Joystick fizyczny + sterowanie z API
│   │   ├── Watering.cpp/.h # L298N, PWM, fail-safe pompy
│   │   └── DisplayMenu.cpp/.h # Menu LCD 16x2 (I2C)
│   └── WATIR-ESP/        # [DEPRECATED] most WiFi dla starszej rewizji
│
├── docs/
│   ├── API_PROTOCOL.md   # Standard protokołu JSON API
│   └── CHANGELOG.md       # Historia rozwoju firmware
│
└── tools/offline_config/ # Kalibracja offline (Python/Tkinter/pyserial)

```

Rysunek 7 Struktura repozytorium

Katalog tools/offline_config zawiera pomocniczą aplikację GUI (Tkinter) umożliwiającą kalibrację progów bezpośrednio przez port szeregowy, z pominięciem warstwy sieciowej - przydatną na etapie testów jednostkowych komponentów sprzętowych.

5. Protokół komunikacji (REST API)

Wymiana danych pomiędzy stacją bazową, backendem a aplikacją mobilną odbywa się w formacie JSON, zgodnie ze sztywnym kontraktem danych ustalonym na wczesnym etapie prac zespołu (Issue #9: „Kontrakt danych - ustalenie struktury JSON”), którego celem było zapewnienie spójności formatu niezależnie od tego, który członek zespołu pracował aktualnie nad daną warstwą systemu.

5.1. Telemetria (Urządzenie → Backend)

Endpoint: POST /api/telemetry - Arduino wysyła cyklicznie, co 30 sekund, pełny zrzut stanu systemu:

```

{
  "device_id": "WATIR_01",
  "sensors": {
    "temp": 24.5,
    "humidity": 46,
    "soil_moisture_1": 35,
    "soil_moisture_2": 52,
    "water_level_cm": 9
  },
  "status": {
    "water_error": false,
    "pump_active": false,
    "pump_power": 70
  },
  "servos": { "pan": 90, "tilt": 45 }
}

```

Rysunek 8 Przykładowy payload

Backend, po odebraniu pakietu, zapisuje rekord w tabeli telemetry_logs i asynchronicznie wyzwała procedurę retencji danych (opisaną w rozdziale 8). Endpoint GET /api/telemetry zwraca historię odczytów z bezpiecznym limitem maksymalnie 500 rekordów na zapytanie, wykorzystywanym przez aplikację mobilną do rysowania wykresów.

5.2. Sterowanie zdalne (Backend/Aplikacja → Urządzenie)

Metoda	Endpoint	Opis
POST	/api/move	Sterowanie serwami - tryb ciągły (kierunek) lub absolutny (kąt 0–180°)
POST	/api/config	Konfiguracja profilu rośliny (próg wilgotności, interwał, kąty, moc pompy)
POST	/api/water	Szybkie, wymuszone podlewanie (domyślnie ok. 3 s, przy aktualnej mocy)
POST	/api/pump	Pełne sterowanie pompą - moc PWM, czas trwania, odczyt statusu

Tabela 2 Endpointy HTTP obsługiwane przez wbudowany serwer Arduino (port 80).

Sterowanie ciągłe (wirtualny joystick w aplikacji) wysyła komendy kierunkowe:

```
{"direction": "lewo"} // lub "prawo", "gora", "dol", "stop"
{"axis": "X", "value": 120} // sterowanie absolutne, oś "X"/"Y", zakres 0-180°
```

Rysunek 9 Komendy kierunkowe

Endpoint pompy przyjmuje zarówno zapytania o status, jak i komendy wykonawcze, z jawnym zabezpieczeniem dopuszczalnych zakresów mocy i czasu trwania:

```
// Sprawdzenie statusu:
{"action": "status"} → {"status": "success", "pump_power": 70, "pump_active": false}

// Podleż z parametrami (moc 0-100%, czas 500-30000 ms):
{"power": 50, "duration": 5000}
```

Rysunek 10 Zabezpieczenia pompy

Przed wykonaniem komendy podlewania firmware każdorazowo weryfikuje aktualny poziom wody w zbiorniku - mechanizm fail-safe opisany szerzej w rozdziale 6.

5.3. Zarządzanie profilami roślin (CRUD)

Backend udostępnia pełen zestaw operacji CRUD na profilach roślin przechowywanych w PostgreSQL, pod bazowym adresem /api/plants:

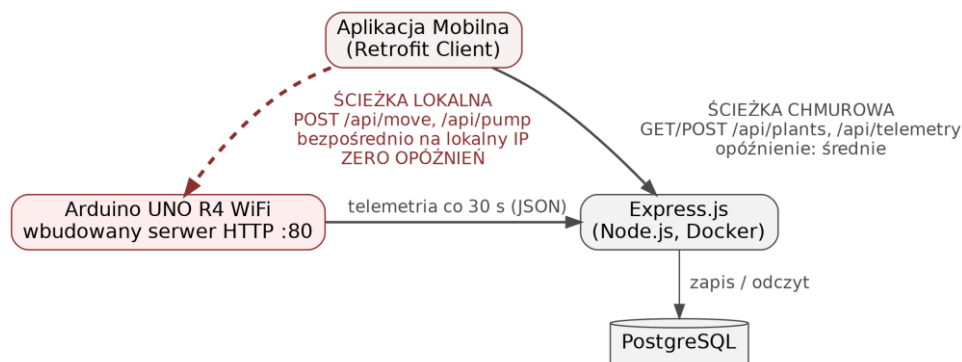
Metoda	Endpoint	Opis
GET	/api/plants	Lista wszystkich profili roślin zapisanych w bazie danych
POST	/api/plants	Utworzenie nowego profilu (nazwa, próg wilgotności, pozycje serw X/Y)
PUT / PATCH	/api/plants/:id	Aktualizacja istniejącego profilu
DELETE	/api/plants/:id	Usunięcie profilu
POST	/api/plants/:id/apply	Pobranie profilu z PostgreSQL i wysłanie konfiguracji do Arduino (/api/config)

Tabela 3 Endpointy zarządzania profilami roślin (CRUD).

Operacja „apply” stanowi kluczowy punkt integracji warstwy chmurowej ze sprzętową - z poziomu aplikacji mobilnej (ekran ustawień, rozwijana lista profili) użytkownik wybiera zdefiniowaną wcześniej roślinę, a backend samodzielnie odczytuje jej parametry z bazy danych i przesyła je bezpośrednio pod lokalny adres IP stacji bazowej, nadpisując aktywną konfigurację bez potrzeby ponownego uruchamiania urządzenia.

6. Hybrydowy routing (Dual-Path) i odporność na awarie

Kluczowym elementem architektonicznym systemu WATIR IoT jest hybrydowy model komunikacji (Dual-Path), rozdzielający ruch sieciowy na dwie niezależne ścieżki w zależności od charakteru przesyłanych danych oraz wymagań dotyczących opóźnień, przedstawiony na Rys. poniżej.



Rysunek 11 Hybrydowy routing Dual-Path: ścieżka chmurowa (telemetria, opóźnienie średnie) i ścieżka lokalna (sterowanie na żywo)

6.1. Ścieżka chmurowa (telemetria i konfiguracja)

Stacja bazowa automatycznie, w cyklu co 30 sekund, generuje i wysyła ładunek JSON zawierający komplet danych telemetrycznych na endpoint API serwera Express.js. Dane są zapisywane w PostgreSQL, skąd aplikacja mobilna odczytuje historyczne wykresy oraz zdefiniowane profile roślin. Ze względu na pośrednictwo chmury (oraz asynchroniczny charakter zapisu i retencji), ścieżka ta charakteryzuje się średnim opóźnieniem - akceptowalnym dla danych o charakterze monitoringowym, nie zaś interwencyjnym.

6.2. Ścieżka lokalna (sterowanie bezpośrednie)

Komendy wymagające natychmiastowej reakcji - sygnały z wirtualnego joysticka 2D oraz żądania natychmiastowego podlewania - celowo omijają warstwę chmurową. Aplikacja mobilna komunikuje się bezpośrednio z lokalnym adresem IP stacji Arduino w obrębie tej samej sieci Wi-Fi, wykorzystując jej wbudowany serwer HTTP. Rozwiązanie to gwarantuje zerowe opóźnienia transmisyjne związane z pośrednictwem chmury, natychmiastową reakcję serwo mechanizmów układu Pan-Tilt oraz pełną niezależność funkcji sterowania manualnego od dostępności połączenia internetowego. Dla porównania obu trybów sterowania serwo mechanizmami sporządzono poniższe zestawienie.

Cecha	Joystick fizyczny	Aplikacja mobilna
Mechanizm	analogRead() → strefa martwa → krok $\pm 5^\circ$	Przeciągnięcie → POST /api/move
Interwał odpytywania	co 25 ms	co 50 ms
Charakter wartości	Inkrementalna ($\pm 5^\circ$ od aktualnej)	Absolutna (0–180°) lub ciągła (kierunek)
Strefa martwa	400–600 (zakres analogowy 0–1023)	0,2 (ułamek pełnego wychylenia)

Tabela 4 Porównanie trybów sterowania serwo mechanizmami ramienia Pan-Tilt.

Ruch suwaka w aplikacji wymaga dodatkowo mechanizmu throttle/debounce (limit ok. 5 pakietów/s), aby nie przeciążyć wbudowanego serwera HTTP mikrokontrolera przy płynnym, ciągłym przeciąganiu - wymaganie to zweryfikowano w ramach Issue #31 („Test opóźnień - Latency Test z Joystickiem”).

6.3. Odporność na błędy i praca w trybie offline (Edge Autonomy)

System zaprojektowano zgodnie z zasadą projektowania odpornego na awarie (fail-safe design). W przypadku utraty łączności zarówno z siecią Wi-Fi, jak i z serwerem chmurowym, stacja bazowa zachowuje 100% autonomii działania

lokalnego - ostatnio zapamiętana konfiguracja oraz progi wilgotności odczytywane są z nieulotnej pamięci EEPROM mikrokontrolera, co umożliwia bezpieczną, samodzielną kontrolę stanu roślin bez konieczności stałego połączenia sieciowego. Mechanizm ten zweryfikowano empirycznie w ramach testu ciągłości opisanego w rozdziale 10.

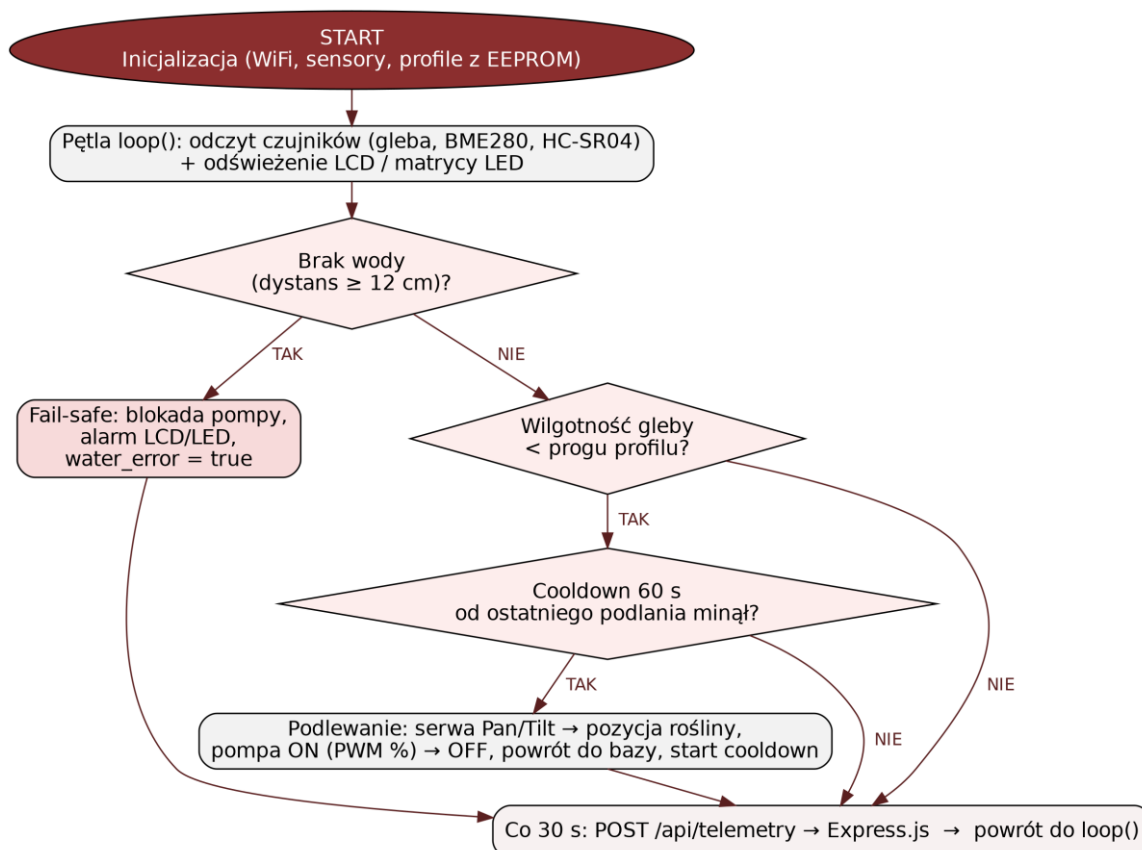
⚠ UWAGA: Endpointy HTTP stacji bazowej nie są obecnie zabezpieczone uwierzytelnianiem - są dostępne dla każdego urządzenia w tej samej sieci lokalnej. Świadomie zrezygnowano również z modułów autentykacji na poziomie backendu (logowanie, tokeny JWT, tabele użytkowników) na rzecz maksymalnego uproszczenia wdrożenia domowego - system działa jako otwarta platforma w sieci prywatnej/domowej.

7. Główne funkcjonalności i logika sterująca

System oferuje trzy komplementarne tryby działania: automatyczne monitorowanie i podlewanie, sterowanie zdalne oraz zapis telemetry wraz z analityką historyczną.

7.1. Tryb automatyczny (AUTO)

W trybie podstawowym mikrokontroler cyklicznie (co ok. 1 sekundę) odczytuje wilgotność gleby dla obu obsługiwanych roślin, temperaturę i wilgotność otoczenia oraz poziom wody w zbiorniku. Jeżeli wilgotność gleby danej rośliny spadnie poniżej progu zdefiniowanego w jej profilu (`moisture_threshold`), serwomechanizmy Pan/Tilt automatycznie pozycjonują ramię nad wskazaną doniczką (kąty z profilu), a pompa uruchamia się na zadany czas z określoną mocą PWM, po czym ramię powraca do pozycji bazowej. Pełną logikę decyzyjną głównej pętli programu, wraz z mechanizmem zabezpieczającym przed pracą „na sucho” oraz cooldownem między cyklami podlewania (60 s, chroniącym przed „powodzią”), przedstawiono na schemacie blokowym poniżej.



Rysunek 12 Schemat blokowy logiki sterującej trybu AUTO - od odczytu czujników, przez zabezpieczenie fail-safe, do decyzji o podlaniu.

Mechanizm fail-safe ma nadrzędny priorytet nad logiką podlewania: jeżeli czujnik HC-SR04 wykryje odległość do lustra wody równą lub większą niż 12 cm (co odpowiada pustemu zbiornikowi w warunkach testowych projektu), system natychmiast blokuje pompę, zapala alarm na LCD i matrycy LED oraz zgłasza flagę `water_error = true` w telemetry - niezależnie od tego, jak bardzo przesuszona jest gleba.

7.2. Sterowanie zdalne (aplikacja mobilna / joystick)

Aplikacja mobilna umożliwia ręczne pozycjonowanie ramienia poprzez wirtualny joystick 2D (kierunki lub wartości absolutne kątów) oraz wymuszenie natychmiastowego podlewania przyciskiem w interfejsie. Systemem można sterować także bezpośrednio przy urządzeniu - za pomocą fizycznego joysticka analogowego z przyciskiem zatwierdzającym, którego naciśnięcie pozwala zapisać bieżącą pozycję ramienia bezpośrednio do pamięci nieulotnej, na wypadek pracy bez towarzyszącej aplikacji.

7.3. Zapis telemetrii i analityka historyczna

Co 30 sekund Arduino wysyła pakiet JSON z bieżącym stanem systemu na serwer backendu. Aby zapobiec niekontrolowanemu rozrostowi bazy danych na serwerach deweloperskich, po każdym zapisie backend automatycznie uruchamia asynchroniczną procedurę retencji (`enforceRetention`, Issue #22), pozostawiając maksymalnie 1000 najświeższych pomiarów na dane urządzenie. Aplikacja mobilna pobiera historię telemetrii i renderuje wykresy temperatury oraz wilgotności gleby i powietrza w czasie, wraz z naniesionymi znacznikami wykonanych cykli podlewania - funkcjonalność tę zaimplementowano i zweryfikowano w ramach Issue #29 („Wykresy telemetryczne i historia zdarzeń”).

8. Baza danych

Warstwa persystencji oparta jest na relacyjnej bazie danych PostgreSQL, skonteneryzowanej wraz z serwerem aplikacyjnym za pomocą Docker Compose i inicjalizowanej skryptem `init.sql`. Logiczny model danych, przedstawiony na Rys. poniżej, obejmuje dwie główne tabele.

telemetry_logs	plant_profiles
id SERIAL PK device_id VARCHAR "timestamp" TIMESTAMPTZ temp REAL humidity REAL soil_moisture_1 REAL soil_moisture_2 REAL water_level_cm REAL water_error BOOLEAN pump_active BOOLEAN pump_power SMALLINT servo_pan SMALLINT servo_tilt SMALLINT <i>idx: (timestamp DESC), (device_id), (device_id, timestamp)</i>	id SERIAL PK name VARCHAR moisture_threshold SMALLINT (0-100) auto_watering BOOLEAN check_interval_ms INTEGER pan SMALLINT CHECK 0-180 DEFAULT 90 tilt SMALLINT CHECK 0-180 DEFAULT 90 pump_power SMALLINT DEFAULT 70

Rysunek 13 Logiczny model bazy danych PostgreSQL - tabele `telemetry_logs` oraz `plant_profiles` wraz z indeksami.

8.1. Tabela `telemetry_logs`

Przechowuje pełną historię odczytów sensorycznych: temperaturę i wilgotność powietrza, wilgotność gleby dla obu roślin, poziom wody w zbiorniku, flagę błędu braku wody, stan i moc pracy pompy oraz aktualne położenie serwomechanizmów Pan-Tilt w stopniach - wszystko powiązane ze znacznikiem czasu i identyfikatorem urządzenia. Zestaw trzech indeksów (chronologiczny malejący po znaczniku czasu, po identyfikatorze urządzenia oraz indeks złożony łączący oba kryteria) optymalizuje odpytanie historii pod kątem błyskawicznego rysowania wykresów w aplikacji mobilnej, sprowadzając złożoność zapytań sortujących do $O(\log N)$ zamiast $O(N)$.

8.2. Tabela plant_profiles

Przechowuje profile opieki nad poszczególnymi roślinami: nazwę, próg wilgotności wyrażony w procentach (0–100%), flagę trybu automatycznego podlewania, interwał sprawdzania w milisekundach oraz docelowe, fizyczne współrzędne ramienia - kąty pan i tilt, ograniczone więzłem CHECK do zakresu 0–180 stopni, z wartością domyślną 90° (pozycja neutralna, środkowa). Rozszerzenie schematu bazy o fizyczne współrzędne serw pozwala, aby każda roślina miała w chmurze zapisaną nie tylko preferowaną wilgotność, lecz również dokładną pozycję przestrzenną głowicy nawadniającej - co jest bezpośrednią konsekwencją przejścia na architekturę 1:Wielu opisaną w rozdziale 1.

9. Środowisko lokalne - instrukcja uruchomienia

9.1. Wymagania wstępne

- Android Studio (najnowsza stabilna wersja) - do budowy i uruchomienia aplikacji mobilnej.
- Docker oraz Docker Compose - do uruchomienia warstwy backendowej i bazy danych.
- Arduino IDE (wersja 2.x zalecana) wraz z paczką płytek Arduino UNO R4 Boards oraz bibliotekami wymienionymi w rozdziale 3.5.
- Aktywne połączenie z siecią Wi-Fi 2,4 GHz, do której podłączona zostanie stacja bazowa.

9.2. Uruchomienie infrastruktury backendowej

1. Sklonuj repozytorium projektu:

```
git clone https://github.com/BaDaQu/WATIR_IoT.git
cd WATIR_IoT/backend
```

2. Utwórz plik zmiennych środowiskowych na podstawie szablonu (dane bazy danych, port):

```
cp .env.example .env
```

3. Uruchom cały stos chmurowy (serwer Express.js oraz baza PostgreSQL) pojedynczą komendą:

```
docker-compose up -d
```

Flaga -d uruchamia kontenery w tle. Docker Compose zbuduje obraz serwera aplikacyjnego z pliku Dockerfile, uruchomi kontener PostgreSQL zainicjalizowany skryptem init.sql oraz skonfiguruje sieć wewnętrzną między kontenerami.

4. Weryfikacja poprawności uruchomienia:

```
docker-compose ps
docker-compose logs -f backend
```

5. Zatrzymanie środowiska:

```
docker-compose down
```

9.3. Instalacja aplikacji mobilnej

- Otwórz katalog app/ w Android Studio i poczekaj na synchronizację zależności Gradle.
- Skonfiguruj w warstwie network/APIClients.kt bazowy URL chmurowego API oraz - opcjonalnie - domyślny lokalny adres IP stacji bazowej w sieci Wi-Fi.
- Uruchom aplikację na emulatorze lub fizycznym urządzeniu Android (przycisk Run).

9.4. Wgrywanie firmware na Arduino UNO R4 WiFi

- Otwórz w Arduino IDE plik firmware/WATIR-ARDUINO/WATIR-ARDUINO.ino.
- Zainstaluj wymagane biblioteki poprzez Narzędzia → Zarządzaj Bibliotekami (patrz Tabela w rozdziale 3.5).
- Upewnij się, że w menu Narzędzia → Płytkę wybrano Arduino UNO R4 WiFi.
- Uzupełnij na początku pliku WATIR-ARDUINO.ino dane sieciowe:

```
const char* backendHost = "10.101.29.4"; // adres IP serwera Node.js
const int backendPort = 3000; // port backendu
```

- Podłącz płytkę kablem USB, wybierz port w menu Narzędzia → Port (Serial natywny USB, 115200 bd), a następnie zweryfikuj i wgraj kod (przyciski Weryfikuj oraz Prześlij).
- Po wgraniu, przy użyciu joysticka należy wybrać opcję wyszukiwania dostępnego Wi-Fi. Następnie zaznaczyć docelowe Wi-Fi z listy sieci znalezionych w pobliżu urządzenia. W przypadku nieznalezienia docelowego Wi-Fi należy zresetować układ przyciskiem reset na mikrokontrolerze i powtórzyć poprzednie kroki. Stacja bazowa nawiąże połączenie WiFi (timeout 60 s, DHCP); w przypadku niepowodzenia użytkownik może spróbować zautoryzować połączenie ponownie lub przejść w pełni funkcjonalny tryb manualny (fizyczny joystick), sygnalizując brak WiFi na matrycy LED.

⚠ UWAGA: Zasilanie pompy (12V) musi być podłączone przez sterownik L298N, ze zdjętą zwórką (jumper) z pinu ENA - patrz rozdział 4.2. Nie wolno zasilać pompy bezpośrednio z pinów 5V Arduino.

10. Testowanie i walidacja systemu

Proces weryfikacji prototypu przeprowadzono metodą iteracyjną, analogicznie do wcześniejszej fazy PoC (rozdział 2.1): każdy komponent sprzętowy przetestowano jednostkowo, a następnie zweryfikowano scenariusze integracyjne oraz end-to-end na poziomie całego systemu. Zadania testowe rejestrowano i śledzono jako osobne zgłoszenia (Issues) w repozytorium projektu.

10.1. Kalibracja czujników

Montaż i kalibrację czujnika gleby, BME280 oraz HC-SR04 (Issue #12) zweryfikowano poprzez odczyt stabilnych wartości na Serial Monitorze Arduino IDE, ustalając wartości progowe ADC dla gleby suchej i mokrej jako punkt wyjściowy do dalszej kalibracji procentowej w profilach roślin.

10.2. Testy integracyjne i end-to-end

Zgłoszenie	Scenariusz	Kryteria akceptacji	Status
Issue #32	Test logiki przypisywania profili - wymuszenie z aplikacji progu „Kaktus” (10%) i weryfikacja jego propagacji do zmiennej threshold na urządzeniu przez port szeregowy.	Płynna zmiana trybu nawadniania w locie, bez restartu urządzenia.	OK
Issue #33	End-to-End Fail-Safe Test - opróżnienie zbiornika, wymuszenie podlewania z aplikacji.	Pompa milczy, błąd zapisany w chmurze, czerwony baner błędu widoczny w aplikacji.	OK
Issue #34	Test Hydrologiczny 48h - urządzenie monitoruje realną doniczkę nieprzerwanie przez dwie doby.	Brak fizycznych przecieków, ramię celnie dozuje wodę, brak utraty łączności WiFi.	OK
Issue #31	Test opóźnień (Latency Test) - sterowanie hybrydowe: aplikacja vs. fizyczny joystick.	Brak „skakania” i „czkawki” silników przy płynnym, zdalnym wymuszaniu ruchu.	OK

Tabela 5 Wybrane scenariusze testów integracyjnych i systemowych.

Test End-to-End Fail-Safe (Issue #33) stanowił krytyczną weryfikację całej pionowej ścieżki systemu - od fizycznego czujnika HC-SR04, przez logikę mikrokontrolera i zapis błędu w chmurze, aż po jego wizualną prezentację w interfejsie mobilnym, potwierdzając spójność mechanizmu bezpieczeństwa na wszystkich trzech warstwach architektury.

10.3. Walidacja stabilności długoterminowej

Test Hydrologiczny 48h (Issue #34) zweryfikował nie tylko poprawność dozowania wody bez zalania otoczenia, lecz również stabilność połączenia WiFi oraz brak zawiesznień pętli programowej mikrokontrolera w warunkach ciągłej, wielogodzinnej pracy - analogicznie do 24-godzinnego testu stabilności przeprowadzonego jeszcze na etapie wersji Proof of Concept (rozdział 2.1).

11. Napotkane problemy i rozwiązania inżynierskie

Integracja poszczególnych warstw systemu - sprzętowej, sieciowej, bazodanowej i aplikacyjnej - wiązała się z szeregiem nietrywialnych problemów inżynierskich. W poniższej tabeli zestawiono dziewięć najistotniejszych z nich wraz z zastosowanymi rozwiązaniami; stanowią one, obok samego kodu źródłowego, jeden z kluczowych rezultatów poznawczych projektu.

1. Maskowanie adresu IP przez bramę Dockera Problem: Serwer Node.js w izolowanej sieci Dockera błędnie wykrywał IP nadawcy telemetryj jako wewnętrzną bramę kontenera (np. 172.18.0.1) zamiast rzeczywistego adresu stacji bazowej, blokując zwrotne sterowanie (POST /api/move, błąd „fetch failed”, HTTP 500). Rozwiązanie: Arduino samodzielnie odczytuje własny adres metodą WiFi.localIP().toString() i wstrzykuje go do treści JSON telemetryj; backend zapamiętuje adres i wykorzystuje przy przekierowaniu komend.	2. Spadki napięcia i niekontrolowane resety Problem: Jednoczesny start pompy i dwóch serwomechanizmów generował skokowy pobór prądu, powodując spadek napięcia na szynie zasilającej i reset mikrokontrolera. Rozwiązanie: Elektryczne rozdzielenie zasilania logiki (5V z LM2596) od zasilania wykonawczego (pompa i serwa pod 12V/3A) przy zachowaniu wspólnej masy (Common Ground) - p. rozdział 4.2.
3. Drganie serwomechanizmów Problem: Ramię wykonywało niekontrolowane drgania wskutek zakłóceń EM anteny WiFi w sąsiedztwie serw oraz konfliktu przerwań między biblioteką Serial a biblioteką Servo. Rozwiązanie: Fizyczne odsunięcie modułu radiowego od serw oraz programowe uśrednianie odczytów joysticka wraz ze strefą martwą (dead-zone) w firmware.	4. Ryzyko przepełnienia bazy telemetryj Problem: Wysoka częstotliwość próbkowania stopniowo zapychała PostgreSQL, spowalniając zapytania i zacinając wykresy w aplikacji mobilnej. Rozwiązanie: Indeksy czasowe PostgreSQL (złożoność $O(\log N)$) oraz asynchroniczny algorytm retencji enforceRetention (Issue #22), ograniczający historię do 1000 najświeższych rekordów na urządzenie.
5. Awaria magistrali I2C po integracji WiFiS3 Problem: Po migracji na R4 WiFi czujnik BME280 na I2C zwracał NaN po WiFi.begin() - biblioteka WiFiS3 rekonfigurowała piny/magistrale podczas nawiązywania łączności. Rozwiązanie: Zmiana kolejności inicjalizacji okazała się niewystarczająca; docelowo BME280 przeniesiono na niezależną magistralę SPI (Chip Select - pin 10, SCK/MISO/MOSI - piny 13/12/11).	6. „Duchy” na I2C i pływający pin CS Problem: Skaner I2C na R4 wykrywał kilkanaście widmowych urządzeń (adresy 0x76–0x7E) lub nie widział czujnika wcale - na starszym R3 identyczne okablowanie działało poprawnie. Rozwiązanie: Zdiagnozowano rygorystyczny kontroler I2C mikrokontrolera Renesas RA4M1 oraz niepodłączony pin CS modułu BME280; problem wyeliminowano wraz z migracją na SPI (poz. 5).
7. Fałszywe alarmy braku wody Problem: Rozruch serw wywoływał chwilowe spadki napięcia, dające błędne odczyty 0 cm z HC-SR04 i wyzwalające fałszywy alarm suchobiegu pompy. Rozwiązanie: Programowy filtr w firmware odrzucający nagłe wartości zerowe i podtrzymujący ostatni prawidłowy,	8. Wielokrotna definicja obiektu czujnika (C++) Problem: Podział kodu firmware na pliki .ino/.cpp skutkował błędem kompilatora „multiple definition of 'bme'”, blokującym wgranie programu. Rozwiązanie: Zastosowanie słowa kluczowego extern - obiekt czujnika tworzony jednokrotnie w Sensors.cpp, plik główny jedynie się do niego odwołuje.

stabilny odczyt dystansu do lustra wody. Wymiana uszkodzonego sensora.	
9. Zarządzanie repozytorium i bezpieczeństwo poświadczeń Problem: Równoległa praca pięciu osób powodowała konflikty wersji; dane z pliku .env oraz katalogi node_modules omal nie trafiły do publicznego repozytorium. Rozwiązanie: Rygorystyczne reguły .gitignore, szablonowy plik .env.example oraz ujednolicenie środowiska deweloperskiego poprzez pracę w kontenerach Docker.	

Tabela 6 Najważniejsze znalezione problemy oraz rozwiązania

12. Kluczowe innowacje, osiągnięcia i podział prac zespołu

12.1. Innowacje wykraczające poza minimum projektu

- **Pełna autonomia lokalna (Edge Autonomy):** system po utracie łączności z siecią WiFi lub serwerem chmurowym zachowuje 100% autonomii działania lokalnego, korzystając z konfiguracji i progów wilgotności zapamiętanych w nieulotnej pamięci EEPROM.
- **Diagnostyczna matryca LED 8x12:** zamiast polegać na podglądzie portu szeregowego, wykorzystano wbudowany panel LED Arduino R4 WiFi - wyświetla płynną animację rosnącej rośliny w normalnym trybie pracy, a w przypadku awarii dzieli ekran na cztery dedykowane sektory diagnostyczne (m.in. utrata WiFi, błąd inicjalizacji BME280, brak wody w zbiorniku, tryb pracy ręcznej).
- **Zgodność z projektowaniem uniwersalnym (WCAG):** interfejs aplikacji mobilnej zaprojektowano z myślą o osobach starszych lub niedowidzących - wysoki kontrast, powiększone pola dotykowe oraz jednoznaczna, czerwona sygnalizacja błędów (np. alarm braku wody).
- **Hybrydowy routing Dual-Path:** świadome połączenie bezpośrednich, bezłagowych komend HTTP w sieci lokalnej z asynchronicznym, niezależnym zapisem historii do chmury (rozdział 6).
- **Pełny cykl wytwórczy (Design Thinking):** projekt zrealizowano od badania rzeczywistych potrzeb rynkowych, poprzez projektowanie obwodów i mechaniki (Pan-Tilt), aż po backend chmurowy i natywny frontend mobilny - z udokumentowaną, iteracyjną ewolucją architektury (rozdział 2).

12.2. Podział prac w zespole

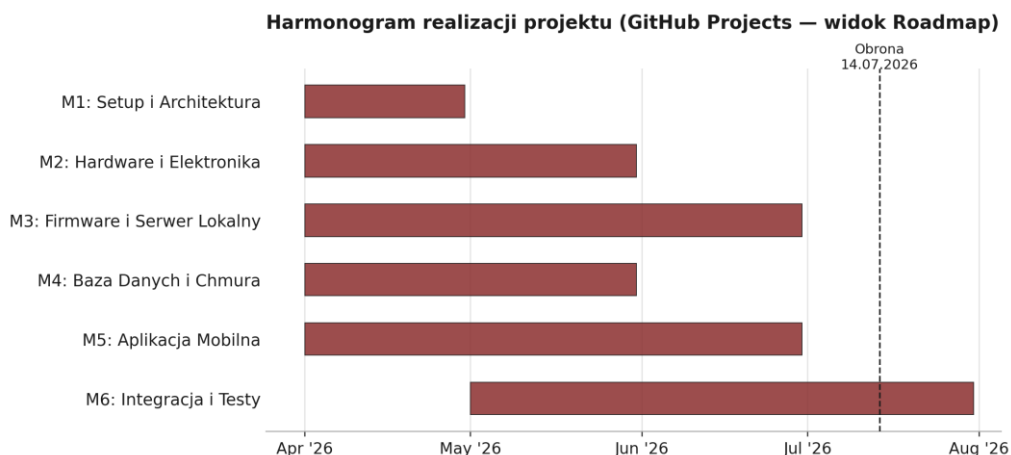
Członek zespołu	Obszar	Zakres odpowiedzialności
Bartłomiej Raj	Lider / Sieci	Zarządzanie projektem (Agile), konfiguracja routingu sieciowego, integracja komponentów chmurowych z firmware, testy integracyjne i ciągłości (m.in. test hydrologiczny 48h).
Wojciech Stochmiatek	Hardware	Projektowanie obwodów zasilania (separacja mas, LM2596), fizyczne złożenie ramienia Pan-Tilt, kalibracja joysticka, montaż i izolacja obudowy przed wilgocią.
Marcel Podlecki	Mobile UI/UX	Makiety w Figma, spójny motyw wizualny aplikacji, implementacja warstwy UI w Jetpack Compose (Dashboard, ustawienia, joystick).
Dominik Pokorski	Mobile Logic	Logika komunikacyjna (MVVM, Retrofit), obsługa asynchronicznych wątków wirtualnego joysticka, wdrożenie wykresów telemetrycznych w czasie rzeczywistym.
Bartłomiej Jedyk	Backend / DB	Projekt struktury bazy PostgreSQL, implementacja API REST (Express.js), konteneryzacja środowiska Docker Compose, asynchroniczna retencja danych.

Tabela 7 Podział odpowiedzialności w zespole projektowym.

13. Harmonogram projektu i zarządzanie zadaniami

Narzędziem koordynacji prac zespołu było GitHub Projects, wykorzystywane w dwóch uzupełniających się widokach: Roadmap (Gantt) - do planowania osi czasu, zależności i monitorowania postępu kamieni milowych, oraz widoku tabelarycznego - do kategoryzacji zadań, przypisywania ich do członków zespołu i śledzenia stanu realizacji (To Do / In Progress / Review / Done). Poszczególne zadania rejestrowano jako indywidualne zgłoszenia (Issues), obejmujące m.in.

konfigurację repozytorium, kontrakt danych JSON, montaż i kalibrację sprzętu, testy end-to-end oraz przygotowanie prezentacji obrończej.



Rysunek 14 Harmonogram realizacji projektu w ujęciu kamieni milowych (widok Roadmap, GitHub Projects).

13.1. Kamienie milowe

- **M1 - Setup i Architektura (kwiecień):** opracowanie kontraktu danych JSON i struktury bazy danych.
- **M2 - Hardware i Elektronika (kwiecień–maj):** kalibracja czujników i budowa konstrukcji Pan-Tilt.
- **M3 - Firmware i Serwer Lokalny (kwiecień–czerwiec):** sterowanie serwami i serwer HTTP; w trakcie tego etapu podjęto decyzję o migracji z systemu dwupłytkowego na jednoukładowe Arduino R4 WiFi.
- **M4 - Baza Danych i Chmura (kwiecień–maj):** konfiguracja API i Dockera.
- **M5 - Aplikacja Mobilna (kwiecień–czerwiec):** implementacja UI i logiki komunikacji.
- **M6 - Integracja i Testy (maj–lipiec):** globalne testy E2E, eliminacja opóźnień, walidacja fail-safe i ostateczny montaż obudowy.

Elastyczność przyjętego narzędzia (GitHub Projects) ułatwiła dynamiczną zmianę założeń architektonicznych w trakcie trwania Milestone 3 bez naruszenia ciągłości prac pozostałych podzespołów - decyzja o migracji sprzętowej została odzwierciedlona w strukturze zadań na bieżąco, a nie poprzez rewizję całego harmonogramu.

14. Znane ograniczenia i możliwości dalszego rozwoju

14.1. Znane ograniczenia

- Zasilacz o wydajności 3A stanowi praktyczny limit systemu - przy pełnej mocy pompy (100% PWM, ok. 2A) w połączeniu z resztą układu istnieje ryzyko brownoutu, dlatego domyślną moc ograniczono do 70%.
- Czujnik BME280 nie może być podłączony magistralą I2C na Arduino UNO R4 WiFi - biblioteka WiFiS3 rekonfiguruje tę magistralę; jedynym stabilnym rozwiązaniem jest komunikacja SPI (rozdział 11.5).
- Piny SDA/SCL są trwale zajęte przez wyświetlacz LCD - nie należy do nich podłączać żadnych innych urządzeń I2C.
- Przy niskiej mocy PWM (ok. 30%) silnik pompy nie ma wystarczającego momentu rozruchowego i jedynie emituje słyszalne buczenie; praktyczne minimum to ok. 40–50% mocy.
- Endpointy HTTP stacji bazowej nie są zabezpieczone uwierzytelnianiem - są dostępne dla każdego urządzenia w tej samej sieci lokalnej (świadomy kompromis na rzecz prostoty wdrożenia domowego).

14.2. Możliwości dalszego rozwoju

- **Integracja Smart Home:** wdrożenie protokołu MQTT i integracja z Home Assistant, umożliwiające pełne włączenie stacji do istniejących ekosystemów automatyki domowej.

- **Uczenie maszynowe:** inteligentne dopasowywanie dawek wody na podstawie historycznych trendów zmian wilgotności powietrza i temperatury rejestrowanych przez BME280.
- **Skalowanie sprzętowe:** rozbudowa stacji bazowej o dedykowany układ elektrozaworów, umożliwiający stabilną i wydajną obsługę większej liczby roślin rozmieszczonych na większych dystansach.
- **Efektywność energetyczna:** tryb Deep Sleep mikrokontrolera z cyklicznym wybudzaniem, wydłużający czas pracy w konfiguracjach zasilanych bateryjnie.
- **Uwierzytelnianie API:** wprowadzenie warstwy autoryzacji dla wdrożeń w sieciach współdzielonych z innymi użytkownikami/urządzeniami.

15. Podsumowanie i wnioski końcowe

Projekt WATIR IoT zrealizował w pełni funkcjonalny, bezpieczny i zintegrowany system Internetu Rzeczy, łączący mikrokontroler klasy Edge, chmurę obliczeniową oraz natywną aplikację mobilną w spójną całość. Zastosowany hybrydowy podział komunikacji - lokalny joystick dla natychmiastowej reakcji oraz asynchroniczna telemetria chmurowa dla danych historycznych - okazał się rozwiązaniem optymalnym pod kątem redukcji opóźnień przy jednoczesnym zachowaniu pełnej historii środowiskowej.

Udokumentowana w rozdziale 2 wieloetapowa ewolucja architektury - od jednoukładowego prototypu laboratoryjnego, przez pośrednie iteracje dwuukładowe i manualne, aż po docelową platformę na Arduino UNO R4 WiFi - stanowi istotną wartość poznawczą projektu, pokazując proces podejmowania decyzji inżynierskich w odpowiedzi na realne ograniczenia sprzętowe. Zespół zyskał praktyczne doświadczenie w integracji niskopoziomowych ograniczeń sprzętowych (magistrale komunikacyjne, budżet mocy, zakłócenia elektromagnetyczne) z nowoczesnymi technologiami chmurowymi i mobilnymi, a napotkane i udokumentowane w rozdziale 11 problemy inżynierskie stanowią, obok samego działającego prototypu, jeden z kluczowych rezultatów kształcących projektu.

16. Literatura i źródła

- Repozytorium kodu źródłowego projektu WATIR IoT: https://github.com/BaDaQu/WATIR_IoT
- Dokumentacja Arduino UNO R4 WiFi: <https://docs.arduino.cc/hardware/uno-r4-wifi/>
- Dokumentacja biblioteki WiFi3 (Arduino): <https://docs.arduino.cc/libraries/wifi3/>
- Bosch Sensortec - dokumentacja czujnika BME280: <https://www.bosch-sensortec.com/products/environmental-sensors/humidity-sensors-bme280/>
- Dokumentacja środowiska Node.js: <https://nodejs.org/en/docs>
- Dokumentacja frameworka Express.js: <https://expressjs.com/>
- Dokumentacja Docker oraz Docker Compose: <https://docs.docker.com/>
- Dokumentacja PostgreSQL: <https://www.postgresql.org/docs/>
- Dokumentacja Android Developers / Kotlin: <https://developer.android.com/kotlin>
- Dokumentacja Jetpack Compose: <https://developer.android.com/jetpack/compose>
- Dokumentacja biblioteki Retrofit (Square): <https://square.github.io/retrofit/>
- Web Content Accessibility Guidelines (WCAG): <https://www.w3.org/WAI/standards-guidelines/wcag/>
- Dokumentacja GitHub Projects: <https://docs.github.com/en/issues/planning-and-tracking-with-projects>
- Specyfikacja protokołu MQTT: <https://mqtt.org/>
- Dokumentacja L298N: <https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>
- Dokumentacja przetwornicy step-down LM2596: https://botland.com.pl/index.php?controller=attachment&id_attachment=1213